

## 2교시: 좋은 Dockerfile 작성 원칙

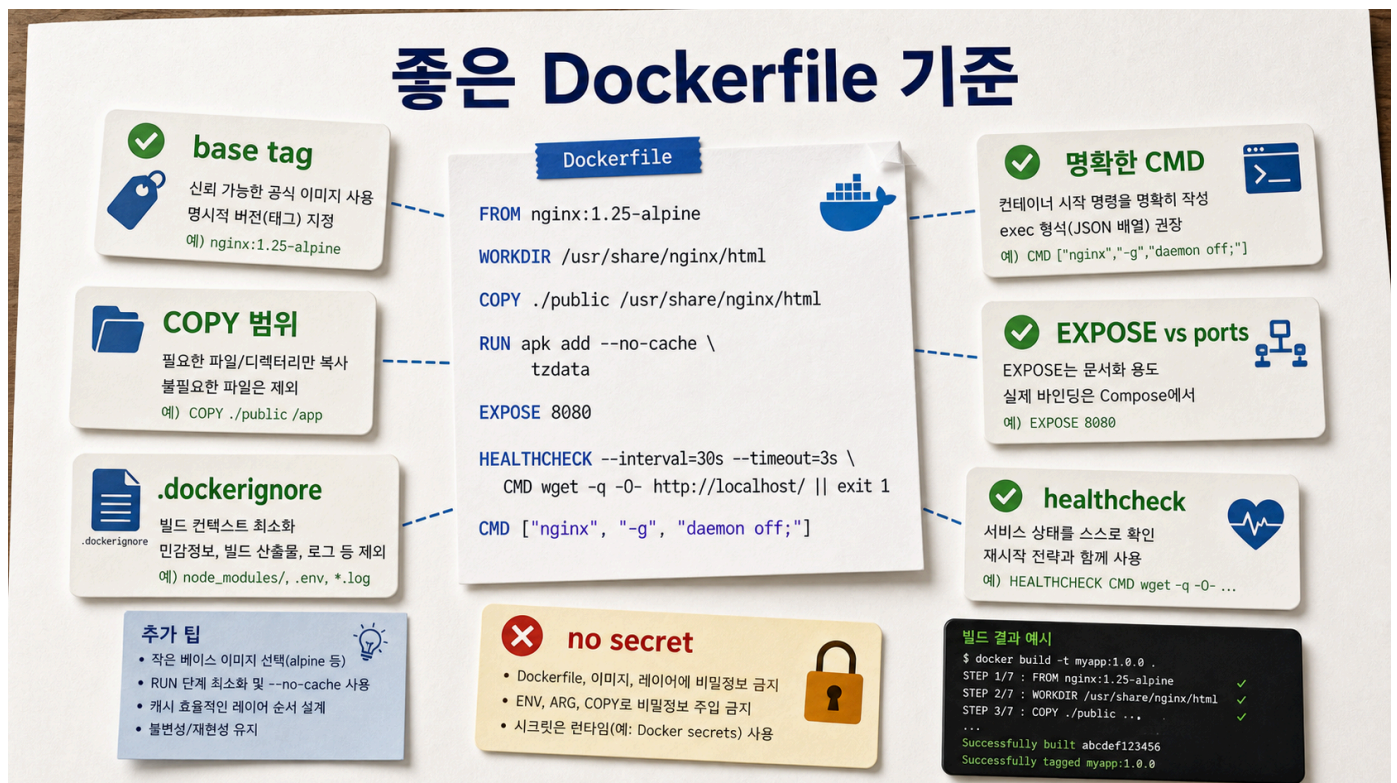
### 수업 목표

- Dockerfile을 build recipe가 아니라 운영 artifact 기준으로 읽는다.
- 작은 image, 명확한 실행 명령, .dockerignore, secret 제외 기준을 설명한다.
- Day 5 통합 앱 Dockerfile을 review하고 개선 가능성을 찾는다.

### 50분 흐름

| 시간     | 활동                           | 비중     | 산출                |
|--------|------------------------------|--------|-------------------|
| 0-8분   | Dockerfile 역할 복습             | 설명 15% | build/runtime 구분  |
| 8-20분  | 좋은 Dockerfile 기준             | 설명 25% | checklist         |
| 20-32분 | 통합 앱 Dockerfile review       | 실행 25% | review note       |
| 32-42분 | .dockerignore와 build context | 실행 20% | context risk note |
| 42-50분 | 평가 루브릭 작성                    | 설명 15% | Dockerfile score  |

### Visual 1: 좋은 Dockerfile 기준



이 visual은 base image, copy 범위, 실행 명령, healthcheck, .dockerignore, secret 제외를 하나의 review board로 보여준다.

### 핵심 설명

Dockerfile은 image를 만드는 절차다. 하지만 Day 5에서는 문법보다 운영 품질을 본다. 좋은 Dockerfile은 작은 image를 만들고, 필요한 파일만 포함하며, 실행 명령이 명확하고, secret을 image layer에 남기지 않는다.

정적 nginx 앱의 Dockerfile은 짧다. 짧다고 쉬운 것이 아니라, 짧은 파일에서도 base image tag, COPY 범위, healthcheck, build context 제외 기준을 확인해야 한다.

## 실습 Dockerfile

```
FROM nginx:1.27-alpine

COPY html/ /usr/share/nginx/html/

EXPOSE 80

HEALTHCHECK --interval=10s --timeout=3s --retries=3 \
  CMD wget -q -O /dev/null http://127.0.0.1/ || exit 1
```

## review 기준

| 항목            | 질문                        | 좋은 기준                          |
|---------------|---------------------------|--------------------------------|
| FROM          | base image가 명시적인가         | nginx:1.27-alpine처럼 version 포함 |
| COPY          | 필요한 파일만 복사하는가             | app artifact만 복사               |
| RUN           | 불필요한 layer를 만들지 않는가       | 필요한 경우에만 사용                    |
| CMD           | 실행 process가 명확한가          | base image default 이해          |
| EXPOSE        | container 내부 port를 문서화하는가 | host publish와 혼동하지 않음          |
| HEALTHCHECK   | 정상 상태를 관찰하는가              | protocol check 포함              |
| .dockerignore | context를 줄이는가             | secret과 불필요 파일 제외              |

## .dockerignore 의 의미

.dockerignore 는 image 안에 들어갈 파일만이 아니라 build context로 Docker daemon에 보내지는 파일 범위를 줄인다. 이 차이가 중요하다.

나쁜 상황:

```
project/
.env
id_rsa
screenshots/
node_modules/
html/
```

.dockerignore 가 없으면 실수로 불필요하거나 위험한 파일이 build context에 포함될 수 있다.

## build context 확인

```
cd week2/day5/labs/integration-app
sed -n '1,120p' .dockerignore
docker build -t paperclip/week2-day5-integration:local .
```

기록할 것:

```
## Dockerfile Review
- Base image:
```

- Copied files:
- Excluded files:
- Runtime port:
- Healthcheck:
- Secret risk:

## 학술 기준 연결

Bloom taxonomy 기준으로 이 교시는 분석과 평가다. 학생은 Dockerfile을 그대로 실행하는 것이 아니라 instruction별 운영 효과를 분석하고 secret/build context 위험을 평가한다.

NIST NICE 관점에서는 configuration과 credential hygiene이 포함된다. Dockerfile review는 보안 review의 가장 초급 단계다.

## 실무 failure mode

| Failure mode    | 증상                         | 예방                    |
|-----------------|----------------------------|-----------------------|
| secret COPY     | image layer에 credential 남음 | .dockerignore, review |
| 너무 큰 context    | build 느낌, 불필요 파일 포함        | context 최소화           |
| latest base만 사용 | 재현성 낮음                     | version tag           |
| healthcheck 없음  | 실행 상태만 보고 정상 오해            | protocol check        |
| shell script 의존 | 실행 경로 불명확                  | CMD/ENTRYPOINT 명시     |

## 오해 점검

| 오해                             | 교정                                           |
|--------------------------------|----------------------------------------------|
| Dockerfile이 짧으면 review가 필요 없다  | 짧아도 base, copy, secret, health를 봐야 한다        |
| .dockerignore는 image size만 줄인다 | build context와 secret risk도 줄인다              |
| EXPOSE가 host port를 연다          | metadata일 뿐 host publish는 -p나 Compose ports다 |
| healthcheck가 있으면 완전한 모니터링이다    | local readiness evidence일 뿐이다                |

## 평가 기준

| 기준            | 2점 evidence                       |
|---------------|-----------------------------------|
| Dockerfile 읽기 | instruction별 의미를 설명했다             |
| context 관리    | .dockerignore 위험을 설명했다            |
| 보안            | secret이 image에 들어가면 안 되는 이유를 설명했다 |
| 운영성           | healthcheck와 HTTP check를 연결했다     |

## 전이 과제

자기 Dockerfile을 아래 기준으로 채점한다.

| 항목       | 0  | 1      | 2           |
|----------|----|--------|-------------|
| base tag | 없음 | latest | version tag |

|           |       |       |                   |
|-----------|-------|-------|-------------------|
| copy 범위   | 전체 복사 | 일부 정리 | 필요한 artifact만     |
| secret 제외 | 없음    | 주의 문구 | .dockerignore와 확인 |
| check     | 없음    | run만  | HTTP/log evidence |

## 공식 근거 링크

- Dockerfile reference: <https://docs.docker.com/reference/dockerfile/>
- Build best practices: <https://docs.docker.com/build/building/best-practices/>

## Dockerfile review worksheet

### ## Dockerfile Review Worksheet

- Dockerfile path:
- Base image:
- Base tag:
- Files copied:
- Files excluded by .dockerignore:
- Runtime port:
- Healthcheck:
- Secret risk:
- Improvement:

## 운영 기준으로 본 instruction

| Instruction | 운영 질문                                         |
|-------------|-----------------------------------------------|
| FROM        | 신뢰할 수 있고 재현 가능한 base인가                        |
| WORKDIR     | 실행 경로가 명확한가                                   |
| COPY        | 필요한 artifact만 포함하는가                           |
| RUN         | build dependency와 runtime dependency가 섞이지 않는가 |
| ENV         | secret을 고정하지 않는가                              |
| EXPOSE      | 내부 port 문서화가 되어 있는가                           |
| CMD         | container primary process가 명확한가               |
| HEALTHCHECK | 최소 정상 확인이 가능한가                                |

## Dockerfile review worksheet

### ## Dockerfile Review Worksheet

- Dockerfile path:
- Base image:
- Base tag:
- Files copied:
- Files excluded by .dockerignore:
- Runtime port:
- Healthcheck:
- Secret risk:
- Improvement:

## build cache와 layer 이해

Dockerfile instruction은 layer를 만든다. 모든 instruction이 같은 비용을 갖는 것은 아니다. source file 변경이 자주 일어나는 instruction을 어디에 두는지에 따라 cache 효율이 달라질 수 있다.

Day 5의 nginx 정적 앱은 단순하지만, 이후 node/python app으로 넘어가면 dependency install layer와 source copy layer를 구분하는 것이 중요해진다.

| Dockerfile 판단                       | 이유                               |
|-------------------------------------|----------------------------------|
| dependency install을 source copy와 분리 | source 변경 때 dependency cache 재사용 |
| 불필요한 file COPY 금지                   | image size와 secret risk 감소       |
| explicit base tag                   | 재현성 향상                           |
| build output만 copy                  | runtime image 단순화                |

## security review와 Dockerfile

Dockerfile review는 보안 review의 일부다.

| 질문                         | 위험                 |
|----------------------------|--------------------|
| .env가 COPY되는가              | credential leak    |
| SSH key가 context에 있는가      | account compromise |
| package manager cache가 남는가 | image bloat        |
| root로만 실행하는가               | privilege risk     |
| unknown base image인가       | supply chain risk  |

## 운영 기준으로 본 instruction

| Instruction | 운영 질문                                         |
|-------------|-----------------------------------------------|
| FROM        | 신뢰할 수 있고 재현 가능한 base인가                        |
| WORKDIR     | 실행 경로가 명확한가                                   |
| COPY        | 필요한 artifact만 포함하는가                           |
| RUN         | build dependency와 runtime dependency가 섞이지 않는가 |
| ENV         | secret을 고정하지 않는가                              |
| EXPOSE      | 내부 port 문서화가 되어 있는가                           |
| CMD         | container primary process가 명확한가               |
| HEALTHCHECK | 최소 정상 확인이 가능한가                                |

## Dockerfile 점검 prompt

```
## Dockerfile Review Note
- 가장 명확한 instruction:
- 가장 위험해 보이는 instruction:
- build context 우려:
```

- secret 노출 가능성:
- README에 추가할 설명: